Relatório — Laboratório 2: limites do processo (heap e stack)

Disciplina: Sistemas Operacionais · TADS/IFRN · 2026.1
Aluno(a): _____
Data: ____/____/_____

> Os endereços e medidas de memória variam conforme arquitetura, compilador e sistema. Nos experimentos abaixo, os números identificados como exemplo são os apresentados no enunciado; o experimento do desafio foi executado neste workspace.

Previsões

1. **Mapa de memória:** o código e a string literal devem ficar em regiões próximas, assim como as globais inicializada e zerada; a variável local deve ficar na stack e o bloco de `malloc()` no heap, portanto afastados das regiões do programa.
2. **400 alocações sem escrita:** espero que `VmSize` aumente em aproximadamente 400 MiB, mas `VmRSS` continue baixo, pois os blocos reservados não foram tocados.
3. **Limite de memória virtual:** espero que `malloc()` retorne `NULL` quando o processo alcançar o teto de `ulimit -v`, permitindo ao programa tratar a falha.
4. **Recursão infinita:** espero que o limite configurado para a stack seja atingido antes de acabar a memória física da máquina; o processo deve receber `SIGSEGV`.
5. **Dobrar o limite da stack:** espero que a profundidade máxima de recursão aproximadamente dobre, mas continue terminando em `SIGSEGV`.
6. **Arquivo de 10 GB:** não é necessário ter 10 GB de RAM se o arquivo for processado em blocos pequenos e reutilizados.

Parte 1 — Regiões de memória

As globais inicializada e zerada ficam em regiões de dados do executável (`.data` e `.bss`); podem ter endereços vizinhos, mas são seções distintas. O código fica em `text`, a string literal em `rodata`, a alocação pequena de `malloc()` normalmente no `[heap]`, e a variável local na `[stack]`. Os endereços concretos devem ser conferidos na execução local em `/proc/<PID>/maps` — não são fixos entre execuções.

`VmSize` contabiliza o espaço de endereçamento virtual reservado pelo processo. `VmRSS` contabiliza páginas residentes na memória física naquele momento. `VmData` é a memória classificada pelo kernel como dados do processo e `VmStk` informa o tamanho da stack.

Partes 2 e 3 — Crescimento e limite do heap

No modo sem escrita, o `malloc()` reserva endereços virtuais, mas páginas físicas não precisam ser materializadas até que o processo as acesse. Por isso, no exemplo do enunciado, após reservar 400 MiB, `VmSize` chega a cerca de 413 MB e `VmRSS` permanece em cerca de 3 MB.

Ao escrever em cada bloco, o processo passa a usar as páginas; no exemplo de 120 MiB do enunciado, `VmRSS` acompanha `VmSize` (aproximadamente 116 MB). O tamanho e o comportamento exatos dependem do ambiente.

Com `( ulimit -v 262144; ./heap 2048 )`, o limite de 256 MiB vale apenas para o subshell e seus processos. O exemplo do enunciado mostra `malloc()` falhando no pedido do MiB nº 253: o programa verifica o retorno, imprime a mensagem e termina com código 1. Isso é diferente de deixar o processo ultrapassar o limite do container e depender do comportamento do OOM killer.

Partes 4 e 5 — Crescimento e limite da stack

Cada chamada de `explode()` cria um quadro de ativação com endereço de retorno,

parâmetros e variáveis locais. A recursão cresce em direção a endereços menores nesta plataforma até atingir o limite da stack. Com `ulimit -s` em 8192 KiB, o exemplo do enunciado imprime nível 7500 e termina com `SIGSEGV` (código 139, isto é, 128 + 11). Ao aumentar o limite para 16384 KiB, o último nível impresso passa a aproximadamente 15500. A profundidade real é um pouco maior que o último múltiplo impresso.

O aviso do compilador sobre recursão infinita não impede a compilação: é um diagnóstico, não uma violação que torne o programa inválido em C. A recursão é intencional neste experimento, e `-O0` ajuda a evitar que a otimização transforme a chamada em um laço.

Parte 6 — Processamento de arquivo

As estratégias incremental e inteira calculam a mesma soma, mas usam quantidades diferentes de memória. A incremental reutiliza um buffer de 4 KiB, portanto seu consumo de memória de trabalho permanece praticamente constante conforme o arquivo cresce. A estratégia inteira aloca um bloco do tamanho do arquivo; para um arquivo de 40 GB em uma máquina com 8 GB de RAM, ela não é viável como estratégia geral e pode falhar na alocação ou provocar pressão de memória. Eu usaria processamento incremental (streaming), tratando cada bloco antes de ler o próximo.

Comparação final: heap e stack

Característica	Heap	Stack
Finalidade	Dados com tamanho ou duração definidos durante a execução	Chamadas de função, parâmetros e variáveis locais automáticas
Crescimento	Gerenciado pelo alocador; pode usar `brk` e/ou `mmap`	Quadros são adicionados e removidos conforme chamadas e retornos
Quem solicita	O programa solicita ao alocador (`malloc()`); o alocador obtém memória do kernel	Convenção de chamada e código gerado usam a região de stack já provida pelo sistema
Exemplo	Estrutura dinâmica ou buffer cujo tamanho é conhecido em runtime	Variável local de uma função
Liberação	Normalmente explícita com `free()`	Automática ao retornar da função
Limite	Espaço virtual, limites do processo/sistema e disponibilidade do alocador	Limite de stack do processo, configurável com `ulimit -s`
Falha possível	`malloc()` retorna `NULL` quando não consegue atender ao pedido	Estouro pode gerar `SIGSEGV`
Manifestação	Falha explícita, que o programa pode testar e tratar	Falha ao acessar a página que excede a região válida; o processo recebe um sinal

Perguntas de investigação

1. **Por que `VmSize` chegou a 413 MB com `VmRSS` em 3 MB?** O processo reservou um grande intervalo de endereços virtuais, mas não escreveu nos blocos. Assim, podia endereçá-los, embora quase nenhuma página correspondente precisasse estar residente na RAM.
2. **Por que os limites falham de formas diferentes?** `malloc()` é uma solicitação explícita: pode detectar a impossibilidade e retornar `NULL`. A stack é usada implicitamente por chamadas e variáveis locais; ao ultrapassar seu limite, o acesso à memória não pode ser atendido e o kernel sinaliza o processo com `SIGSEGV`.
3. **Por que o compilador não impede a recursão infinita?** O padrão da linguagem permite recursão e o compilador normalmente apenas emite um aviso. A execução pode depender de entrada/condição; não é obrigação do compilador rejeitar esse código.
4. **Por que `.data` e `.bss` são separadas mesmo com endereços vizinhos?** Elas têm semânticas diferentes: `.data` contém valores inicializados que precisam ser carregados; `.bss` representa dados inicialmente zerados. Separá-las permite ao executável e ao carregador tratar inicialização, tamanho e mapeamento

corretamente. Proximidade no espaço de endereços não significa que pertençam à mesma seção.
5. **O que aconteceria se heap e stack crescessem na mesma direção?** Poderiam consumir o espaço disponível um do outro mais cedo, limitando o crescimento de uma região mesmo quando a outra ainda tivesse pouca utilização. Crescer em direções opostas permite compartilhar o espaço de endereçamento disponível com mais flexibilidade.
6. **Como processar um CSV de 40 GB em uma máquina com 8 GB de RAM?** Usaria leitura incremental/streaming, por exemplo, linha a linha ou em blocos. Carregar tudo de uma vez requer memória proporcional ao arquivo e não cabe, além de precisar considerar memória adicional para estruturas de processamento.

Desafio — liberar cada bloco

Foi adicionado ao `heap.c` o modo `./heap 8 sim liberar`, que toca cada bloco e chama `free()` logo em seguida. A execução de teste neste workspace mostrou:

- Inicial: `VmSize=2696 kB`, `VmRSS=1748 kB`, `VmData=224 kB`.
- Final: `VmSize=3720 kB`, `VmRSS=2900 kB`, `VmData=1248 kB`.

Neste teste, o `VmSize` **não** retornou ao valor inicial. Após os primeiros blocos, as medidas ficaram aproximadamente estáveis; isso é compatível com o alocador mantendo memória em sua arena/cache para reutilização, em vez de devolver imediatamente todas as páginas ao kernel. O resultado depende da implementação do alocador e do ambiente, então não é uma regra universal.

Para testar essa hipótese, pode-se comparar as chamadas de memória do processo com `strace` (`brk`, `mmap`, `munmap`) e observar `/proc/<PID>/maps` durante o experimento. Também se pode testar `malloc_trim(0)` (extensão da glibc) após as liberações e comparar `VmSize`/`VmRSS` antes e depois; se a memória retida for devolvida, os valores devem diminuir. O `free()` torna o bloco disponível para reutilização pelo alocador, mas não garante por si só que o processo reduzirá imediatamente o espaço mapeado.

Questões finais

1. **Diferença entre heap e stack:** o heap sustenta alocações dinâmicas cuja vida útil é controlada pelo programa; a stack sustenta chamadas de função e dados locais, liberados automaticamente no retorno.
2. **Diferença entre `VmSize` e `VmRSS`:** `VmSize` é espaço virtual total contabilizado; `VmRSS` é a parcela atualmente residente em memória física.
3. **Relação entre `malloc()` e heap:** `malloc()` é uma interface do alocador para pedir memória dinâmica. O alocador pode obter essa memória do kernel por diferentes mecanismos; nem todo `malloc()` corresponde necessariamente à região `[heap]` mostrada em `maps`.
4. **Relação entre chamadas e stack:** uma chamada normalmente cria um quadro de ativação na stack; o retorno remove esse quadro.
5. **Por que recursão excessiva gera `SIGSEGV`?** Cada chamada consome mais stack. Ao ultrapassar a região permitida, a próxima referência não pode ser atendida e o kernel sinaliza o processo. Diferente de `malloc()`, chamadas comuns não retornam um valor `NULL` informando que o quadro não coube.
6. **Por que um arquivo de 10 GB não exige 10 GB de RAM?** Se for processado em blocos, só é necessário manter na memória o bloco atual e o estado do cálculo, não o arquivo inteiro.
7. **Por que o sistema operacional controla recursos dos processos?** Para limitar e contabilizar recursos, proteger a máquina e permitir que programas reajam a limites. No experimento do heap, `ulimit -v` força uma falha controlável de `malloc()`; no da stack, `ulimit -s` limita a recursão e demonstra que um processo não pode crescer indefinidamente.

Observação de compilação

Os quatro programas compilam com `gcc -Wall -Wextra -O0`. `pilha.c` emite o aviso `infinite recursion detected`, esperado para a recursão intencional do

experimento.